

VisionX API Design

README-style technical plan for the FastAPI backend, PostgreSQL schema, camera streaming flow, recording flow, and dashboard endpoints.

1. Stack Lock-In

Use this stack and avoid changing it unless a real limitation appears.

Frontend: React + Vite + Tailwind CSS
Backend: FastAPI + Python
Database: PostgreSQL
Live viewing: go2rtc
Recording: FFmpeg
Storage: NAS-mounted folders for MP4 files
Jobs/queue: Redis + Python worker
Deployment: Docker Compose
Reverse proxy: Caddy
Remote access: Tailscale
Monitoring: Prometheus + Grafana
Mobile: PWA first, Expo later
AI: OpenCV first, ONNX/TensorFlow Lite later

2. Core Idea

FastAPI should control the system, not process every video frame itself. Video-heavy work belongs to go2rtc, FFmpeg, and worker processes.

Camera RTSP stream

- > go2rtc for live viewing
- > FFmpeg worker for recording
- > NAS stores .mp4 files
- > PostgreSQL stores metadata
- > FastAPI exposes APIs
- > React/PWA displays dashboard
- > AI workers create events later

3. Service Responsibilities

Service	Responsibility
visionx-web	React dashboard for live view, recordings, events, settings, and health.
visionx-api	FastAPI backend for auth, camera CRUD, recording metadata, health, events, and file URLs.
visionx-worker	Runs FFmpeg jobs, health checks, thumbnails, cleanup, and later AI detection.
visionx-postgres	Stores metadata only. Do not store video files in the database.
visionx-redis	Queue for background jobs.
visionx-go2rtc	Restreams RTSP cameras for browser/mobile live viewing.
NAS	Stores MP4 recordings, snapshots, thumbnails, and event images.

4. MVP Scope

- Login with one admin account.
- Add, update, delete, and test RTSP cameras.
- View live camera feed in the web dashboard.
- Record fixed-length MP4 clips to the NAS using FFmpeg.
- Save recording metadata in PostgreSQL.
- List and play recordings from the dashboard.
- Show online/offline camera health status.
- Run everything with Docker Compose.

5. Database Schema

Start with these tables only. Add automation, alerts, and face recognition later.

users

users

- id uuid primary key
- email text unique not null
- password_hash text not null
- display_name text
- role text default 'admin'
- created_at timestampz
- updated_at timestampz

cameras

cameras

- id uuid primary key
- name text not null
- location text
- rtsp_url text not null
- stream_key text unique not null
- enabled boolean default true
- recording_enabled boolean default true
- health_check_enabled boolean default true
- created_at timestampz
- updated_at timestampz

recordings

recordings

- id uuid primary key
- camera_id uuid references cameras(id)
- start_time timestampz not null
- end_time timestampz
- duration_seconds integer
- file_path text not null
- file_size_bytes bigint
- format text default 'mp4'
- status text default 'recording'
- has_motion boolean default false
- has_person boolean default false
- created_at timestampz

status values:

- recording

- completed
- failed
- deleted

camera_health_checks

camera_health_checks

- id uuid primary key
- camera_id uuid references cameras(id)
- checked_at timestamptz
- status text not null
- latency_ms integer
- last_frame_at timestamptz
- last_recording_at timestamptz
- error_message text

status values:

- online
- offline
- degraded
- recording_failed
- unknown

events

events

- id uuid primary key
- camera_id uuid references cameras(id)
- recording_id uuid references recordings(id)
- event_type text not null
- start_time timestamptz not null
- end_time timestamptz
- confidence numeric(5,4)
- label text
- description text
- snapshot_path text
- metadata jsonb
- created_at timestamptz

event_type examples:

- motion_detected
- person_detected
- vehicle_detected
- face_detected
- camera_offline
- recording_failed
- storage_warning

snapshots

snapshots

- id uuid primary key
- camera_id uuid references cameras(id)
- event_id uuid references events(id)
- recording_id uuid references recordings(id)
- captured_at timestamptz not null
- file_path text not null
- file_size_bytes bigint
- created_at timestamptz

6. Relationship Diagram

Camera

- > many Recordings
- > many Health Checks
- > many Events
- > many Snapshots

Recording

- > many Events
- > many Snapshots

Event

- > optional Recording
- > optional Snapshot
- > later Alerts / Face Detections

7. API Base

Base URL: /api/v1

Auth: HttpOnly cookie session or JWT cookie

Format: JSON for API requests and responses

Files: Video/snapshot endpoints stream files from NAS or Caddy later

8. Auth Endpoints

POST /api/v1/auth/register

POST /api/v1/auth/login

POST /api/v1/auth/logout

GET /api/v1/auth/me

POST /api/v1/auth/login

Request:

```
{
  "email": "you@example.com",
  "password": "secure-password"
}
```

Response:

```
{
  "user": {
    "id": "uuid",
    "email": "you@example.com",
    "display_name": "Phuc"
  }
}
```

9. Camera Endpoints

POST /api/v1/cameras

GET /api/v1/cameras

GET /api/v1/cameras/{camera_id}

PATCH /api/v1/cameras/{camera_id}

DELETE /api/v1/cameras/{camera_id}

POST /api/v1/cameras/{camera_id}/test

GET /api/v1/cameras/{camera_id}/stream

POST /api/v1/cameras

Request:

```
{
  "name": "Front Door",
  "location": "Porch",
  "rtsp_url": "rtsp://user:pass@192.168.1.50:554/stream1",
  "stream_key": "front_door",
  "recording_enabled": true
}
```

Response:

```
{
  "id": "camera-uuid",
  "name": "Front Door",
  "location": "Porch",
  "stream_key": "front_door",
  "enabled": true
}
```

GET /api/v1/cameras

Response:

```
[
  {
    "id": "camera-uuid",
    "name": "Front Door",
    "location": "Porch",
    "stream_key": "front_door",
    "enabled": true,
    "status": "online",
    "last_seen_at": "2026-06-27T10:00:00Z"
  }
]
```

POST /api/v1/cameras/{camera_id}/test

Response:

```
{
  "camera_id": "camera-uuid",
  "status": "online",
  "latency_ms": 83,
  "message": "RTSP connection successful"
}
```

GET /api/v1/cameras/{camera_id}/stream

Response:

```
{
  "camera_id": "camera-uuid",
  "stream_url": "/live/front_door",
  "stream_type": "webrtc"
}
```

Note:

FastAPI returns the stream URL. go2rtc serves the live stream.

10. Recording Endpoints

```
GET /api/v1/recordings
GET /api/v1/recordings/{recording_id}
GET /api/v1/recordings/{recording_id}/video
POST /api/v1/cameras/{camera_id}/recordings/start
POST /api/v1/cameras/{camera_id}/recordings/stop
```

GET /api/v1/recordings

Query params:

- camera_id optional
- date optional, example 2026-06-27
- status optional

Example:

GET /api/v1/recordings?camera_id=uuid&date=2026-06-27

Response:

```
[
  {
    "id": "recording-uuid",
    "camera_id": "camera-uuid",
    "camera_name": "Front Door",
    "start_time": "2026-06-27T10:00:00Z",
    "end_time": "2026-06-27T10:05:00Z",
    "duration_seconds": 300,
    "file_size_bytes": 42000000,
    "has_motion": false,
    "has_person": false,
    "video_url": "/api/v1/recordings/recording-uuid/video"
  }
]
```

POST /api/v1/cameras/{camera_id}/recordings/start

Response:

```
{
  "job_id": "job-uuid",
  "camera_id": "camera-uuid",
  "status": "queued"
}
```

11. Health Endpoints

GET /api/v1/health/cameras

GET /api/v1/cameras/{camera_id}/health

GET /api/v1/health/cameras

Response:

```
[
  {
    "camera_id": "camera-uuid",
    "camera_name": "Front Door",
    "status": "online",
    "latency_ms": 75,
    "last_seen_at": "2026-06-27T10:00:00Z",
    "last_recording_at": "2026-06-27T09:55:00Z"
  }
]
```

12. Event Endpoints

Events can start simple. At first, create camera_offline and recording_failed events. Later, add motion_detected and person_detected from AI workers.

GET /api/v1/events

```
GET /api/v1/events/{event_id}
GET /api/v1/events/{event_id}/snapshot
```

GET /api/v1/events

Query params:

- camera_id optional
- event_type optional
- date optional

Response:

```
[
  {
    "id": "event-uuid",
    "camera_id": "camera-uuid",
    "camera_name": "Front Door",
    "recording_id": "recording-uuid",
    "event_type": "person_detected",
    "start_time": "2026-06-27T22:14:12Z",
    "confidence": 0.9132,
    "snapshot_url": "/api/v1/events/event-uuid/snapshot",
    "recording_url": "/api/v1/recordings/recording-uuid/video"
  }
]
```

13. Later Automation Endpoints

```
POST /api/v1/automation-rules
GET /api/v1/automation-rules
PATCH /api/v1/automation-rules/{rule_id}
DELETE /api/v1/automation-rules/{rule_id}
```

Example automation rule

```
{
  "name": "Front door person alert at night",
  "trigger_type": "person_detected",
  "conditions": {
    "camera_id": "front-door-id",
    "after_time": "22:00",
    "before_time": "06:00"
  },
  "actions": {
    "send_notification": true,
    "channel": "pushover",
    "priority": "high"
  }
}
```

14. Worker Jobs

Job types:

- record_camera_chunk
- check_camera_health
- generate_thumbnail
- analyze_recording_motion
- analyze_recording_objects
- send_alert
- cleanup_old_recordings

Example job:

```
{
  "job_type": "record_camera_chunk",
  "camera_id": "camera-uuid",
  "duration_seconds": 300
}
```

Recording worker flow

1. Query enabled cameras from PostgreSQL.
2. For each camera, start FFmpeg.
3. Save MP4 file to NAS.
4. Check file exists and file size is valid.
5. Insert recording row into PostgreSQL.
6. Queue thumbnail or AI analysis job.

FFmpeg command shape

```
ffmpeg -rtsp_transport tcp -i "rtsp://username:password@192.168.1.50:554/stream1" -t 300 -c copy -y "/mnt/nas/visionx/recordings/front_door/2026/06/27/10-00-00.mp4"
```

15. NAS Folder Layout

```
/mnt/nas/visionx/
  recordings/
    front_door/
      2026/06/27/10-00-00.mp4
      2026/06/27/10-05-00.mp4

  snapshots/
    front_door/
      2026/06/27/event-uuid.jpg

  thumbnails/
    front_door/
      2026/06/27/recording-uuid.jpg
```

16. Build Order

Phase 1 - Camera POC

1. Get RTSP URL working in VLC.
2. Run go2rtc.
3. Show one live stream in the browser.
4. Display stream in React.

Phase 2 - Backend + Database

1. Create FastAPI app.
2. Add PostgreSQL.
3. Add camera CRUD endpoints.
4. Add camera test endpoint.

Phase 3 - Recording

1. Mount NAS into worker container.
2. Run FFmpeg manually.
3. Automate 5-minute recordings.
4. Save recording metadata.
5. Show recordings in React.

Phase 4 - Health

1. Add health worker.
2. Save camera health checks.
3. Show online/offline status.

Phase 5 - Polish MVP

1. Add auth.
2. Add Docker Compose.
3. Add README, screenshots, and architecture diagram.

Phase 6 - ML Features

1. Add motion detection.
2. Add person/object detection.
3. Add event timeline.
4. Add alerts and automation rules.

17. Final MVP Definition

VisionX is a self-hosted security camera dashboard that connects to RTSP cameras, displays live feeds through go2rtc, records 5-minute MP4 clips to NAS storage using FFmpeg, stores metadata in PostgreSQL, and monitors camera online/offline health through FastAPI background workers.

Keep the MVP narrow. The first complete version should do live view, recording, playback, health checks, auth, and Docker deployment. Add ML after that foundation works.